

第一次动手做的东西：料箱抓取 demo 上手指南

整理日期：2026-09-12 目的：2 周内交出"我做过的"，用来换一次面谈

0. 这份东西是什么（一句话）

在仿真里搭一个装杂乱物体的料箱，用深度相机拍它，然后让程序只看深度图找出"夹爪该从哪里夹进去"，并统计成功率。

产出 = 一张 4 联图 + 一组数字。这就是你要递给教授的"我做过的东西"。

1. 先把几个名词变成大白话

- 仿真 (simulation)：在电脑里搭一个假的物理世界（重力、碰撞都是真的算出来的）。好处是不花钱、不坏设备、可以一晚上跑 100 次。你用的是 MuJoCo（机器人领域最常用的物理仿真器之一，DeepMind 维护，免费）。
- 深度图 (depth image)：每个像素存的不是颜色，而是"这个点离相机多远"。工业抓取里机器人就是靠它知道物体的形状。
- 分割图 (segmentation)：每个像素标出"它属于哪个物体"。这份 demo 里它只用来判分（课后答案），不给抓取算法看——这点很重要，说明你的方法是"只用深度"。
- 对跖抓取 / 2D 抓取矩形：平行夹爪要夹住东西，理想情况是两个手指顶在物体两侧的相对平行的面上。在图像上就表现为"一条线 + 一个宽度"，所以学术上叫抓取矩形 (grasp rectangle)。Cornell 抓取数据集就是这么标注的。
- bin picking (料箱拣选)：一箱乱七八糟的零件，机器人自己看到、找到能抓的点、抓出来。这就是你要读的方向里最典型的工业任务。

2. 两条路，选一条（别都做）

	A. 仿真路线（推荐）	B. 真实图像路线
用什么	MuJoCo + 深度图（我已经写好代码）	Cornell Grasping Dataset（真实照片+人工标注的抓取矩形）
你要做什么	跑通 → 改参数 → 跑对比实验 → 写报告	下载数据集 → 训练/复现一个抓取检测模型（需要 GPU 或慢慢跑）
2 周能交吗	能（1 周跑通、1 周做对比）	勉强，容易卡在环境和训练上
适合谁	第一次动手、想先证明"能干活"	已经会 PyTorch、想直接碰学习类方法

你的情况选 A。B 以后想做再说——B 的价值在于"真实数据 + 学习方法"，A 的价值在于"我一周内就能给你看结果"。

3. 我已经给你跑通的东西

文件都在 `D:\Hermes\抓取demo\`：

- `bin\_picking\_demo.py` —— 主程序（约 230 行，中文注释）
- `demo\_result.png` —— 输出图：左上抓取结果 / 右上原图 / 左下深度图 / 右下抓取质量热图
- `demo\_summary.txt` —— 统计结果

我这边实测的结果（30 个随机场景）：

指标	结果
成功选出抓取点的场景	30/30 = 100%
抓取合格（接触点在同一物体上、夹爪路径无遮挡）	27/30 = 90%
平均每场候选抓取数	4.7
单场耗时	约 0.1 秒

跑法（这台机器上依赖我已装好：mujoco 3.13.0 / numpy / matplotlib / pillow）：

```
```bash
cd D:\Hermes\抓取demo
python bin_picking_demo.py 30 # 30 个场景；不填默认 20
```
```

4. 怎么把它变成"你的版本"（这是报告里"结果"的来源）

不要原样交我这份——至少改一个变量，跑一组对比。三个最好改的：

1. 相机高度/角度（`make\_xml` 里的 `pos`、`xyaxes`）：从正上方改成斜视，遮挡变少，成功率会变——这是最像研究的一组实验
2. 夹爪宽度范围（`GRIP\_MIN` / `GRIP\_MAX`）：收窄会漏掉大物体，放宽会夹到不该夹的地方
3. 评分权重（`score = 0.5 \* s\_w + 0.5 \* s\_s`）：改成 0.8/0.2 看结果怎么变

每组改完跑 30 个场景，把 `demo\_summary.txt` 的数字填进一张表 →

这就是你的一页实验结果。表格长这样（示例，数字换成你跑出来的）：

| 实验 | 相机俯角 | 选出率 | 合格率 |
|----|------|------|-----|
| 基线 | 53° | 100% | 90% |
| 更斜 | 40° | ? | ? |
| 更陡 | 75° | ? | ? |

5. "遇到的问题 3 条"怎么写（照挑，别编）

真做过的人才写得出这些——你只要真的跑过，就会遇到：

1. MuJoCo 的 `render()` 每次只返回"最后启用的"那一种图。一开始我以为它会依次返回彩色/深度/分割，结果三种图混成一团（深度图变成了两个通道的分割图）。查了半天才明白要"启用一种 → 渲染 → 再启用下一种"。
2. 相机从上往下看，根本看不到物体侧面，所以最开始那版"找两个法线相对的表面"的方法，一个抓取点都找不出来（候选 0）。想通的关键是：工业里判断能不能夹，看的是物体在某个方向上的宽度能不能塞进夹爪，而不是两个表面的法线。改成"投影宽度"之后立刻有结果了。
3. 深度图的归一化会毁掉对比度：背景是超远值，直接用 min-max 拉伸，整张图变成一片白。改成按 2%~98% 分位拉伸，物体轮廓才出来。

（还可以写：分割图返回的是 (H,W,2)

两通道、接触点必须落在同一物体掩码内否则叫"抓空气"、球体只能抓到可见的那一弧面等。）

6. 一页报告模板（照这个写，别加长）

...

标题：基于深度图的料箱抓取点选择（仿真验证）

1. 背景（2 行）：料箱拣选中，机器人需要从深度信息里自主找抓取位置。

我在 MuJoCo 里搭了一个仿真料箱，用深度相机验证一个最朴素的抓取点选择方法。

2. 方法（3-4 行）：

- 场景：6 个随机摆放的物体（方块/圆柱/球）+ 料箱
- 输入：480×360 深度图（分割图只用于事后判分）
- 算法：对每个物体，遍历 18 个方向 × 5 条扫过线，取"该方向上的投影宽度"

作为夹爪开合宽度，宽度落在 2.5~8.5cm 内即可抓；再按"宽度是否舒服 +

夹持面是否平坦"打分，取最高分，非极大值抑制后输出前 3 个抓取。

- 判定：两个接触点是否落在同一物体上、夹爪路径是否被其他物体挡住

3. 结果（表格 + demo_result.png）

4. 遇到的问题（3 条，见上）

5. 下一步我想怎么做：加入物体位姿估计；用真实深度相机（RealSense）复现；

把抓取点接到机械臂做闭环。

...

7. 两周时间表（每天 1~2 小时）

| 时间 | 做什么 |
|-----------|-----------------------------------|
| Day 1 | 装环境 + 跑通我给的代码（今天就能完成） |
| Day 2-3 | 读一遍代码，把每个函数的作用写进自己的话（报告"方法"段就靠这个） |
| Day 4-7 | 改相机角度，跑 3 组对比实验，记录数字 |
| Day 8-10 | 再改一个变量（夹爪宽度或评分权重），补第二组表 |
| Day 11-12 | 写一页报告 + 截好图 |
| Day 13-14 | 课后当面交给教授，说那三句话 |

8. 交给教授时的话（照抄）

1. 교수님, 지난번에 여쭙본 것 관련해서, 이번엔 제가 직접 해봤습니다.

2. 결과를 한 장으로 정리해 왔는데, 잠깐 봐주실 수 있을까요?

3. 연구실에서 학부연구생으로 배울 수 있을까요? 급여는 바라지 않고, 데이터 정리나 실험 보조부터 하고 싶습니다.

更正：那张对比表里有一个错误（重要，别照抄进报告）

后来我做了失败原因普查（`census.py`，每格 30

个场景）：堆叠场景里失败的原因几乎全是"上方通道被挡"——

30°：23/30、45°：20/30、60°：25/30，"选不出候选"全部为 0。

于是我去核对这个判据本身，用它自己的深度数据打了它的脸：

...

接触点（真正要夹的地方）深度 = 0.864 m

上方 10px：物体 6，深度 0.886 m

上方 20px：物体 1，深度 1.001 m

上方 30px：物体 11，深度 0.989 m

...

上面那些点全都比接触点更远。深度越大 =

离相机越远；它们在画面里"高"，只是因为离得远、在空间里位置更高，

而不是"悬在接触点正上方的障碍物"。所以现在这条"图像上方 40px 通道"的判据是物理上不成立的近似，它把"更远的物体"误判成了"挡路的物体"——之前那张 20~37% 的表就是这么来的。

结论（这条才是能写进报告的）：

- > 单张深度图无法判断夹爪能否从正上方进入抓取位置。
 - > 从相机方向看，能看到的表面就是第一层；至于"正上方的空间有没有被别的物体占着"，
 - > 在图像上根本不是一个可以往上找的问题——需要 3D
- 几何（相机内外参已知，可把像素反投影到世界坐标）
- > 或者第二个视角。

所以下一步（这才是真正的"多做一点"）：用相机内外参把接触点反投影到世界坐标，

再检查"该点正上方的竖直圆柱（半径 3cm、高 5cm）里有没有别的物体"——用真 3D 判据替换现在的假判据，

然后重跑那张对比表。结论可能变，也可能不变，两种都是好结果。

附二：抓取仿真台（能转旋钮、立刻看结果的小程序）

桌面新增图标 抓取仿真台（快捷键 `Ctrl + Alt + S`），或双击文件夹里的 `run\_studio.bat`。

界面：左边一列旋钮，右边是画面，下面是统计。

| 旋钮 | 作用 |
|----------------|------------------|
| 相机俯角 (15~85°) | 越小越斜，越大越接近正上方 |
| 箱内物体数量 (3~20) | 越多越堆叠、越难 |
| 每个参数跑多少场景 | 批量统计的样本数 |
| 夹爪最小/最大张开 (cm) | 决定"多宽的东西能夹" |
| 要求"能从上方下手" | 打开后会排除那些下不去手的抓取点 |

按钮：跑单场（随机一个场景，1 秒内出图） / 跑一批（按设定场景数统计，出百分比） / 保存当前图。

这个工具的用法不是"再跑一遍看看"，是"改一个旋钮，盯着数字变"——这比读代码快十倍地建立直觉。

附三：到底要不要再装一个"真正的仿真软件"？

不用。你已经有一个（MuJoCo），缺的不是引擎，是会用它的手。真要装之前，先看这张表：

| 工具 | 什么时候值得装 | Windows 上的现实 |
|---------------|--------------------------------------|--|
| MuJoCo (现在这个) | 接触丰富的操作、抓取、强化学习、快 | 一行 `pip install`, 已经跑通 |
| Gazebo | 要 ROS 2 完整流水线: URDF、传感器插件、MoveIt、仿真机 | 官方说明 Windows 支持是 experimental, 推荐 Ubuntu |
| Isaac Sim | 要照片级渲染、大规模合成数据、GPU 并行 | 要 NVIDIA 显卡 + 十几 GB, 重 |
| CoppeliaSim | 想要现成 GUI 教具、拖拽式搭场景 | 可装, 但和你现在做的事重叠 |
| RoboDK | 只想做机械臂离线编程 / 轨迹仿真 | 商业软件, 和"研究抓取算法"方向不同 |

判断规则 (记这一条就够) :

> 你要验的是算法想法 → MuJoCo (现在这个) 。

> 你要对接的是实验室的整机系统 → 装 ROS 2 + Gazebo, 在 Ubuntu (WSL2) 里装, 不是 Windows 原生, 而且最好进组之后用组里的环境装。

最后一句实话: 换工具不会让你的报告变好看, 多一组对比数据才会。

9. 诚实的边界 (写进报告, 也写进你心里)

- 这是2D 视觉抓取点选择, 不是完整的 6-DoF 抓取规划, 也不是真机实验
- 分割图 (真值) 只用于判分; 抓取算法本身只用了深度图
- 仿真里的物体形状整齐, 真实料箱有反光/透明/堆叠遮挡——这些是"下一步"要解决的
- 不要把这份东西说成"我做了 bin picking 系统"。年轻教授一眼看得出深浅; 说"我做了个最小验证, 想在这上面继续", 比夸大安全一百倍

附: 对比实验已经跑出来了 (可以直接用)

加了一条更真实的判定条件后 (夹爪必须能从上方空手进去, 不能只是"两点在同一物体上"), 合格率变得有意义, 也真正随视角变化了:

| 场景 | 相机俯角 | 选出抓取 | 抓取合格 | 选出率 | 合格率 |
|--------------|------|-------|-------|------|-----|
| 稀疏 (6 个物体摊开) | 30° | 30/30 | 9/30 | 100% | 30% |
| 稀疏 | 45° | 30/30 | 11/30 | 100% | 37% |
| 稀疏 | 60° | 30/30 | 8/30 | 100% | 27% |
| 堆叠 (12 个物体) | 30° | 30/30 | 3/30 | 100% | 10% |
| 堆叠 | 45° | 30/30 | 6/30 | 100% | 20% |
| 堆叠 | 60° | 30/30 | 3/30 | 100% | 10% |

两条能写进报告的结论

1. 视角不是越陡越好, 也不是越斜越好, 中间 (45°) 最好。太斜 (30°) 时前面的物体会挡住后面的; 太陡 (60°) 时相邻物体在图像上挤在一起, "从上方下手"的通道更容易撞到邻居。
2. 场景越堆叠, 合格率越低 (37% → 20%)。这正是 bin picking 的核心难点: 不是"看不到", 而是"看得到但下不去手"。

注意一个容易搞错的地方: 早先那版 90%+ 只判"两个接触点在同一物体上、闭合路径无遮挡"; 现在加上"上方接近通道"这条真实约束, 数字掉到 27~37%——数字掉是变严格了, 不是变差了。

报告里要写清楚用的是哪套判定标准，这也是审稿/答辩必问的一句。

第三个按钮（桌面：跑角度对比，快捷键`Ctrl + Alt + C`）就是跑这张表的：

双击 → 20 秒 → 自动打开结果表和各角度样例图。